Wireless Security - WiFi performance lab report

Gabriele Trainiti

1 INTRODUCTION

This report details a comparative performance analysis of WiFi networks under controlled stress conditions. The evaluation methodology isolates two primary transport protocols, TCP and UDP, sequentially, while systematically varying the Maximum Segment Size (MSS) and Maximum Transmission Unit (MTU) parameters for each. Testing encompasses three distinct link topologies: wireless-to-wireless (WiFi–WiFi), wired-to-wireless (Ethernet-WiFi), and wired-to-wired (Ethernet–Ethernet) baselines.
Data collection was performed across a heterogeneous testbed comprising multiple WiFi network interface controllers, diverse operating system environments. The following sections present the quantitative results regarding throughput, goodput, and packet loss derived from these stress tests, focusing strictly on the observed correlations between configuration variables and network performance metrics.

2 TOOLS AND THEORY

The Goodput in this study is defined as the rate of useful application layer data delivered to the destination, excluding protocol overhead, retransmissions, and headers. Given the variable nature of WiFi contention and the impact of MSS/MTU tuning, theoretical maximums are adjusted using the following estimation model:

$$G(L) = C_{phy} \cdot \frac{L}{L + OH} \cdot \eta_{TCP/UDP}$$

2.1 TCP

Goodput is expected to peak when MSS aligns with the negotiated MTU minus header overhead, minimizing fragmentation while avoiding excessive retransmission penalties during congestion. Deviations from optimal MSS are anticipated to reduce throughput due to increased header-to-payload ratios or TCP window scaling inefficiencies.
The efficiency factor η is not derived analytically for modern standards (802.11n/ac/ax) because MAC overhead with frame aggregation does not admit a closed-form expression. Instead, it is calibrated empirically on a reference measurement in ideal conditions. Once calibrated, η is held constant to predict the goodput at all other payload sizes. The model is therefore validated, not fitted, against independent measurements. Systematic deviations reveal physical effects that the simplified single-frame model cannot capture:

- **Small frames:** the model may overestimate goodput when the packet rate exceeds the end-host's processing capability (network stack, CPU, or driver limitations).
- **Large frames:** the model may underestimate goodput when frame aggregation (e.g., A-MPDU in 802.11n/ac/ax) amortizes MAC overhead over multiple subframes, pushing the effective efficiency above the single-packet assumption.

Wireless Security Report, Turin, 2026
2026. ACM ISBN .
https://doi.org/

The analysis will focus on different values of MSS:

$$MSS = 1448 \implies G(1448) = C_{phy} \cdot \frac{1448}{1448 + 64} \cdot \eta_{TCP} = C_{phy} \cdot 0.958 \cdot 0.51$$

$$MSS = 700 \implies G(700) = C_{phy} \cdot \frac{700}{700 + 64} \cdot \eta_{TCP} = C_{phy} \cdot 0.916 \cdot 0.51$$

2.2 UDP

As a connectionless protocol, UDP goodput is primarily constrained by the physical layer capacity and the chosen datagram size. Larger MTUs are expected to yield higher goodput by reducing per-packet processing overhead, provided the link remains stable under stress.

For UDP the payload size is controlled directly via the `-l` option in iperf3. The efficiency factor $\eta_{\text{UDP}} \approx 0.82$ is calibrated on the close-range measurement at $L = 1412$ B, where the measured goodput is 712.6 Mbps:

$$\eta_{\text{UDP}} = \frac{712.6}{866 \cdot \frac{1412}{1520}} \approx 0.82$$

Using $H_{\text{tot}} \approx 108$ byte and $\eta_{\text{UDP}} = 0.82$:

$$L = 1412 \implies G(1412) = 866 \cdot \frac{1412}{1520} \cdot 0.82 \approx 866 \cdot 0.762 \approx 660 \text{ Mbps}$$

$$L = 500 \implies G(500) = 866 \cdot \frac{500}{608} \cdot 0.82 \approx 866 \cdot 0.674 \approx 584 \text{ Mbps}$$

$$L = 100 \implies G(100) = 866 \cdot \frac{100}{208} \cdot 0.82 \approx 866 \cdot 0.394 \approx 341 \text{ Mbps}$$

All the measurements were conducted using iperf3[1] (version 3.21) because of its precise timing and detailed JSON output, suitable for automated analysis, as done by this report via the script 1, changing the needed parameters to switch between TCP and UDP.

3 SETUP AND SCENARIOS

The tests were carried out from different setups. The paper will cite the involved workspace for each scenario.

3.1 Workspace 1

Network's default gateway is addressed 192.168.1.1/24

3.1.1 *Server.* Windows 11 @ 192.168.1.11/24

The laptop's network card[2] supports 5GHz bandwidth and 802.11ac, with MCS9 256-QAM, a single spatial stream and long GI, which caps the data rate at 390 Mbps. This relatively poor performance may be related to the sharing of the chip with the Bluetooth module, so the company might have chosen a least performing Wifi interface in order to limit the production cost of the laptop.

3.1.2 *Client.* EndeavourOS (arch Linux) @ 192.168.1.13/24

The client's network card[3] operates in two different ways depending on up/down stream. As a receiver, the max throughput in 802.11ac @ 80MHz with VHT-MCS 7 and two spatial streams (VHT-NSS 2) is estimated to be 585 Mbps with a long GI. As a transmitter, it is instead using VHT-MCS 9 @ 80MHz with short GI and two spatial streams (VHT-NSS 2), which reaches 866.7 Mb/s.

3.2 Workspace 2

3.2.1 Server. Samsung Galaxy S24

The Samsung Galaxy S24 acts simultaneously as Access Point (AP) and iperf3 server. The device is connected to power, background apps are closed, and power saving is disabled. The WiFi link operates on IEEE 802.11ac at 5 GHz (channel 149, 80 MHz bandwidth) with WPA2-Personal security. The negotiated PHY rate in close range is 866 Mbps (MCS 9, 2 spatial streams, short GI 400 ns).

3.2.2 Client. MacBook Air M4

The MacBook Air M4 connects to the AP hosted by the Galaxy S24. The client's network card supports 802.11ac at 5 GHz with 80 MHz channel bandwidth. As a receiver, the max throughput in 802.11ac @ 80 MHz with MCS 9 and two spatial streams (VHT-NSS 2) is estimated to be 866 Mbps with a short GI (400 ns). As a transmitter, it operates with the same configuration, reaching 866 Mb/s. The bottleneck is exclusively the WiFi link; there are no intermediate routers or switches. The complete testbed configuration is summarized in Table 9.

4 RESULTS

4.1 TCP

4.1.1 MTU 1500. Using the setup described in 3.1 and setting the Maximum Transmission Unit to 1500 with

```
ip link set dev wlan0 mtu 1500
```

the results in the case of a wireless-to-wireless connection shows a peculiar behaviour

Table 1: WiFi-Wifi TCP @ 1500 MTU

Goodput	Value
Mean	168 Mbps
Minimum	7.33 Mbps
Maximum	217 Mbps
Standard deviation	35.4 Mbps

With respect to the theoretical maximum physical limit, it is clear that the goodput is lower than the expected value as shown in 2.1 by 43% on average, with a peak of 55% while being perfectly in line with the expected goodput calculated in 2.1. This result shown in Figure 1 may be explained with the WiFi medium being in a congested environment with multiple active devices, which reduces available airtime per station under CSMA/CA. TCP performance is further degraded by rate variability and delay spikes, which limit congestion window growth and sustained sending rate. Even in the absence of significant packet loss, increased MAC-layer backoff and retransmissions reduce effective channel utilization.

Another test was performed over a wired Ethernet connection using a 1 m Cat5e cable between the client device and the AP, eliminating WiFi airtime contention and wireless rate adaptation effects. By doing so, the bottleneck should shift away from the client–AP link and be primarily determined by the server-side cap or intermediate network constraints. In this setup, TCP is expected to achieve throughput values closer to the imposed 390 Mb/s ceiling, assuming no additional congestion or shaping elsewhere in the path.

The results show indeed a better performance, with an average efficiency of 80% of the physical capacity

Table 2: CAT5e-Wifi TCP @ 1500 MTU

Goodput	Value
Mean	315 Mbps
Minimum	69.2 Mbps
Maximum	365 Mbps
Standard deviation	54.1 Mbps

Another experiment was conducted with both the client and the server connected via wired Ethernet links, using a CAT5e cable on the client and a CAT8 cable on the server side. Given that CAT8 supports well above 1 Gb/s, the effective bottleneck of the path is expected to be the CAT5e segment. Therefore, the maximum achievable throughput should approach the physical limit imposed by the weakest link. This result has been proven to be accurate, with an average efficiency of 90%

Table 3: CAT5e-CAT8 TCP @ 1500 MTU

Goodput	Value
Mean	901 Mbps
Minimum	717 Mbps
Maximum	963 Mbps
Standard deviation	60.9 Mbps

4.1.2 MTU 700. Using the WiFi-WiFi setup, an additional TCP test was performed with both client and server connected to the same access point over an 802.11a wireless link.

Table 4: CAT5e-CAT8 TCP @ 700 MTU

Metric	Value
Mean	9.33 Mbps
Minimum	9.08 Mbps
Maximum	9.60 Mbps
Standard deviation	0.16 Mbps
Total retransmissions	0
Average RTT	54.95 ms

The measured goodput is significantly lower than the values obtained with more favourable wireless configurations, but it is also much more stable. The standard deviation is only 0.16 Mbit/s, and no TCP retransmissions were observed in any of the 15 runs. This suggests that the low throughput clearly shown in Figure 3 is not mainly caused by packet losses, but rather by the limitations of the 802.11a link, the WiFi-WiFi airtime sharing, and the TCP window/RTT interaction.

A useful consistency check can be made by considering the approximate bandwidth-delay product limitation. The JSON outputs report a TCP window close to 65280 bytes and an average RTT of approximately 55 ms. Therefore, the expected TCP rate can be estimated as

$$G \approx \frac{W \cdot 8}{RTT} = \frac{65280 \cdot 8}{0.055} \approx 9.5 \text{ Mbit/s}.$$

This value is very close to the measured average goodput of 9.33 Mbit/s. Hence, the results are coherent with a TCP flow limited by the effective window size and the relatively high RTT of the WiFi-WiFi path, rather than by packet loss.

4.2 UDP

Using the setup described in Section 3.2, the UDP scenarios were evaluated in two distinct environments by controlling the application payload size L via the `-l` option in `iperf3`: a close-range scenario under ideal conditions (PHY rate 866 Mbps), and a far-range scenario with severe attenuation caused by physical obstacles (PHY rate dropping to 6-27 Mbps).

4.2.1 MTU 1412. The value of 1412 bytes was chosen because the Android hotspot enforces an effective MTU of 1440 bytes; setting $L_{max} = 1440 - 28 = 1412$ prevents IP fragmentation. At this payload size, the wireless link reaches its maximum efficiency in the close-range scenario, achieving a goodput plateau of 712.6 ± 29.2 Mbps that outperforms the theoretical single-packet model due to 802.11ac A-MPDU aggregation (Fig. 2, left). In contrast, in the far-range scenario, the goodput collapses to a mere 4.93 ± 2.87 Mbps on average due to multipath fading and intermittent link outages (Fig. 2, right). The Wireshark I/O Graph for the close-range test (Fig. 4) confirms a stable flow at ~650 Mbps after an initial ramp-up, with no fragmentation or rate collapses. Conversely, the I/O Graph for the far-range test (Fig. 5) reveals extreme instability, with throughput oscillating between 2.5 and 15 Mbps and frequent drops caused by intermittent link outages.

Table 5: WiFi-Ethernet UDP @ 1412 Payload: Close vs Far Range

Metric	Close Range	Far Range
Mean	712.6 Mbps	4.93 Mbps
Minimum	643 Mbps	1.31 Mbps
Maximum	734 Mbps	9.30 Mbps
Standard deviation	29.2 Mbps	2.87 Mbps

Another experiment was carried out using the setup described in 3.1, as it involves wireless connections both for the client and for the server.

Table 6: WiFi-WiFi UDP @ 1412 Payload

Metric	Close Range
Mean	46.9 Mbps
Minimum	36.9 Mbps
Maximum	54.8 Mbps
Standard deviation	1.68 Mbps

The achieved UDP goodput is significantly lower than the nominal WiFi capacity due to MAC-layer inefficiencies inherent to IEEE 802.11. In addition, the wireless channel was congested, leading to increased contention, collisions, and retransmissions among multiple active devices.

4.2.2 MTU 500. Reducing the payload size to 500 bytes increases the relative impact of the fixed MAC, IP, and UDP headers. In the close-range scenario, this results in a lower average goodput (543.1 Mbps) and a higher variance compared to larger payloads. Interestingly, in the far-range scenario, this payload size yielded a slightly better mean goodput (5.73 Mbps) than the 1412-byte payload, likely due to a lower probability of frame corruption under poor channel conditions.

Table 7: WiFi-Ethernet UDP @ 500 Payload: Close vs Far Range

Metric	Close Range	Far Range
Mean	543.1 Mbps	5.73 Mbps
Minimum	417 Mbps	5.46 Mbps
Maximum	626 Mbps	7.07 Mbps
Standard deviation	63.5 Mbps	0.50 Mbps

4.2.3 MTU 100. With a very small payload of 100 bytes, the theoretical model significantly overestimates the actual measurements. At this size, `iperf3` generates approximately 150,000 packets per second, shifting the bottleneck from the wireless link to the smartphone's CPU and network stack processing capabilities. Consequently, the goodput is heavily capped at 118.5 Mbps in the close-range setup. In the far-range scenario, the goodput falls further to 3.35 Mbps.

Table 8: WiFi-Ethernet UDP @ 100 Payload: Close vs Far Range

Metric	Close Range	Far Range
Mean	118.5 Mbps	3.35 Mbps
Minimum	116 Mbps	2.64 Mbps
Maximum	120 Mbps	3.99 Mbps
Standard deviation	1.2 Mbps	0.59 Mbps

5 CONCLUSIONS

The experiments conducted during the lab activity by our group shows that the theoretical goodput model is useful as a reference, but it cannot fully describe the behaviour of a real WiFi link. In theory, the useful rate should mainly depend on the PHY rate, the payload size, the protocol overhead, and the TCP/UDP efficiency factor. In addition, larger payloads and higher PHY rates should generally produce higher goodput. However, the measurements show that the actual goodput is often significantly lower than the theoretical value because WiFi performance is strongly affected by airtime contention, MAC-layer overhead, rate adaptation, retransmissions, RTT, and end-host processing limitations. This difference is particularly evident in the TCP WiFi-WiFi scenario.

The CAT5e-WiFi configuration achieved a higher average goodput, and the fully wired CAT5e-CAT8 setup reached values close to the Gigabit Ethernet limit. This confirms that the main source of discrepancy between theoretical and measured performance in the WiFi-WiFi case is not the transport protocol alone, but the shared and variable nature of the wireless medium. The TCP test over 802.11a provides a different but important result: the measured goodput was very low, around 9.33 Mbit/s, but it was extremely stable and no retransmissions were observed. In this case, the discrepancy with higher-rate WiFi scenarios is not mainly caused by packet loss, but by the limited 802.11a link and by the TCP window/RTT relationship.

For UDP, the results show that the theoretical model is accurate only under some conditions. In the close-range case with large payloads, UDP reached very high goodput because it does not wait for acknowledgements and does not reduce its rate through congestion control. The measured value can even exceed the simplified single-packet theoretical estimate because modern WiFi mechanisms such as frame aggregation reduce the relative impact of MAC overhead. However, in far-range conditions the goodput collapsed

because UDP does not recover lost packets and is directly exposed to channel degradation.
In conclusion, the theoretical values represent an upper-bound or idealized reference, while the measured goodput reflects the real interaction between physical-layer conditions, MAC-layer access, transport protocol behaviour, and device limitations.

6 APPENDIX

Table 9: Workspace 2 – Testbed configuration.

Parameter	Value
Client	MacBook Air M4, macOS 26.4.1, iperf3 3.21
Server / AP	Samsung Galaxy S24 (Hotspot Mode), Android 16, iperf3 server
WiFi Standard	IEEE 802.11ac, 5 GHz, Ch. 149, 80 MHz
Security	WPA2-Personal
Client IP	10.157.13.132/24
Server IP	10.157.13.192/24

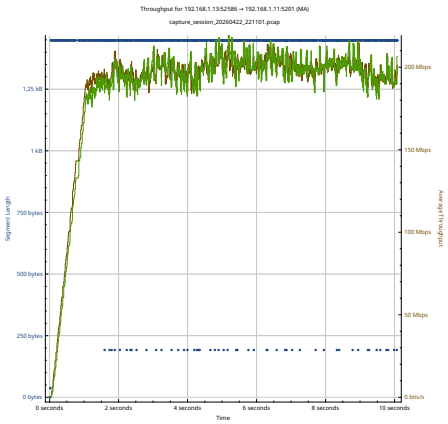


Figure 1: Throughput/Goodput over a TCP conversation with 1448 MSS

Listing 1: Capture script

```
#!/bin/bash

INTERFACE="wlan0"
TARGET_IP="192.168.1.11"
OUTPUT_BASE="capture_session"
TIMESTAMP=$(date +%Y%m%d_%H%M%S)

echo "Starting capture"
sudo tcpdump -i $INTERFACE -s 0 -w
     "./wireshark/udp/${OUTPUT_BASE}_${TIMESTAMP}.pcap" port 5201 &
TCPDUMP_PID=$!

sleep 1

for i in {1..15}; do
  echo "Running test $i of 15"
  iperf3 -c $TARGET_IP -i 1 -J >
       "./iperf_out/udp/iperf_run_${i}_$(date +%Y%m%d_%H%M%S).json"

  sleep 2
done

echo "Stopping capture"
kill $TCPDUMP_PID
wait $TCPDUMP_PID
```

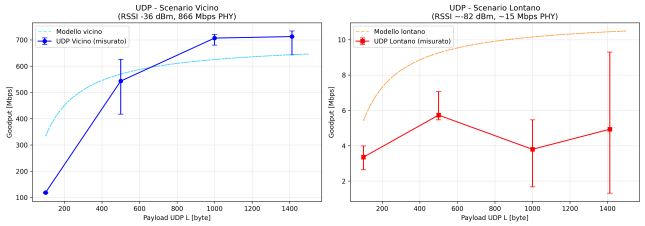


Figure 2: UDP goodput vs payload size. Left: close range (S1, PHY 866 Mbps). Right: far range (S2, PHY ~10 Mbps). Error bars show min/-max over 10 runs.

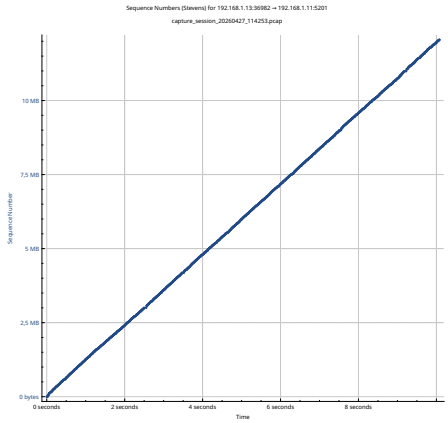


Figure 3: Steven's graph of a TCP conversation on 802.11a and 700 MSS

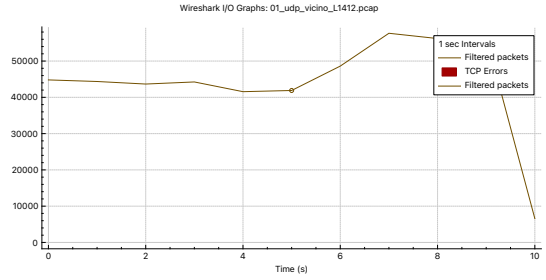


Figure 4: Wireshark I/O Graph for UDP close range ($L = 1412$ B). Throughput remains stable with no drops, confirming the goodput plateau.

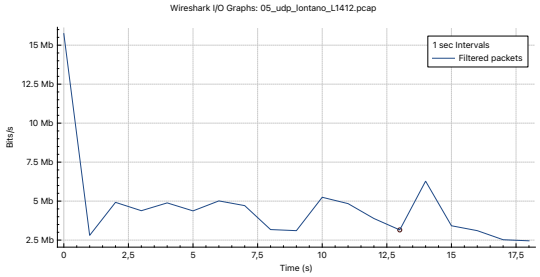


Figure 5: Wireshark I/O Graph for UDP far range ($L = 1412$ B). High variability and intermittent outages are visible.

REFERENCES

[1] *iperf*, https://iperf.fr/iperf-download.php
[2] *Realtek*, https://fcc.report/FCC-ID/2BGCASIV0223/7471505.pdf
[3] *Realtek*, https://www8.hp.com/h20195/v2/GetPDF.aspx/4AA7-2986ENW.pdf